

week9 연습과제

⚙ Status	Not started
≡ Text	10. 비전 트랜스포머 p617~

10 비전 트랜스포머

VIT

- 이미지 인식을 위한 딥러닝 모델
- 이미지를 자연어 처리 방식처럼 분류해 보려는 시도에서 탄생
- CNN - 합성곱 계층 방법(지역 특징 추출) 사용 X
트랜스포머 모델 - 셀프 어텐션(전체 이미지 한 번에 처리) 적용
- 이미지 패치가 왼→오, 위→아래 순차적으로 입력
⇒ 문제) 2차원 구조의 이미지 특성 온전히 반영 X
⇒ 해결) Swin Transformer, CvT 모델 제안

Swin Transformer

- 로컬 윈도우 활용 → 각 계층의 어텐션이 되는 패치의 크기, 개수 다양하게 구성

CvT

- 기존 합성곱 연산 과정을 ViT에 적용한 모델
- low-level feature, high-level feature 계층적으로 반영 가능
- ex) 사람 얼굴
저수준 특징: 눈, 코, 입
고수준 특징: 눈코입을 포함한 전체 얼굴
- 어텐션 연산 과정에서 Query, Key, Value 중 K와 V를 기존 특징 벡터보다 축소 → 계산 복잡도 감소

ViT

- 트랜스포머 구조 자체를 컴퓨터비전 분야에 적용한 첫 번째 연구

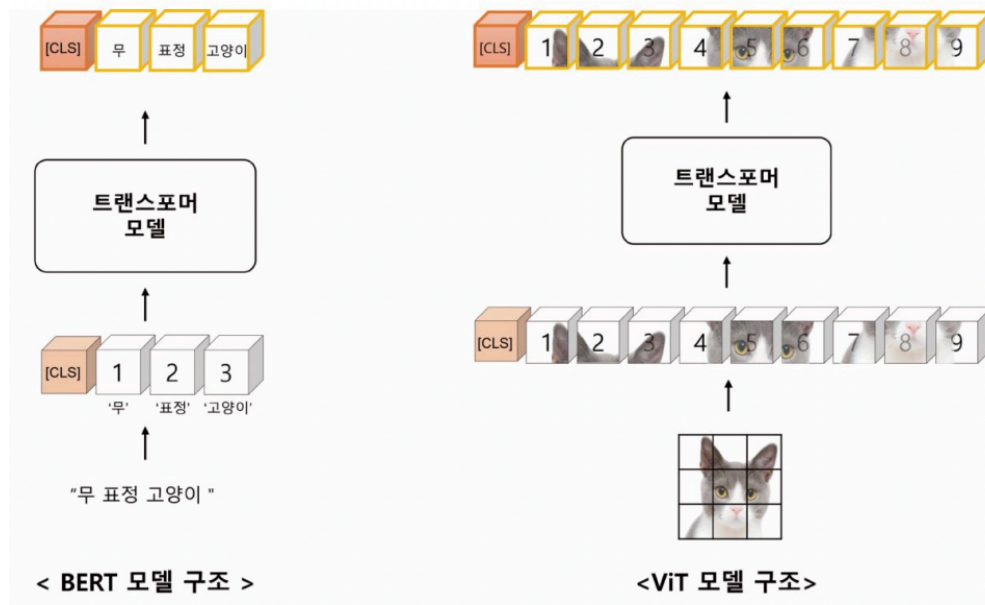


그림 10.2 BERT의 ViT 모델 구조 차이

- 두 모델 다 트랜스포머 모델의 구조 그대로 사용, but 입력 데이터 만드는 과정 다름
- BERT: 문장보다 작은 단위의 토큰 순차적으로 입력
- ViT: 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적 입력
⇒ ViT에 사용되는 입력 이미지 패치는 원→오, 위→아래로 표현된 시퀀셜 배열 가정

합성곱 모델과 ViT 모델 비교

목적: 이미지 특징을 잘표현하는 임베딩 만들기 (공통)

과정에 큰 차이

합성곱 신경망의 임베딩

- 이미지 패치 중 일부만 선택하여 학습 → 이미지 전체 특징 추출
- ex) 고양이 이미지에서 왼쪽 눈의 특징 추출
→ 이미지에서 해당 부분만 선택 학습

ViT 임베딩

- 이미지를 작은 패치들로 나눠 각 패치 간의 상관관계 학습
- 이를 위해 셀프 어텐션 방법 사용 → 모든 이미지 패치가 서로에게 주는 영향 고려 → 이미지 전체 특징 추출
- ⇒ 모든 이미지 패치가 학습에 관여, 높은 수준의 이미지 표현 제공

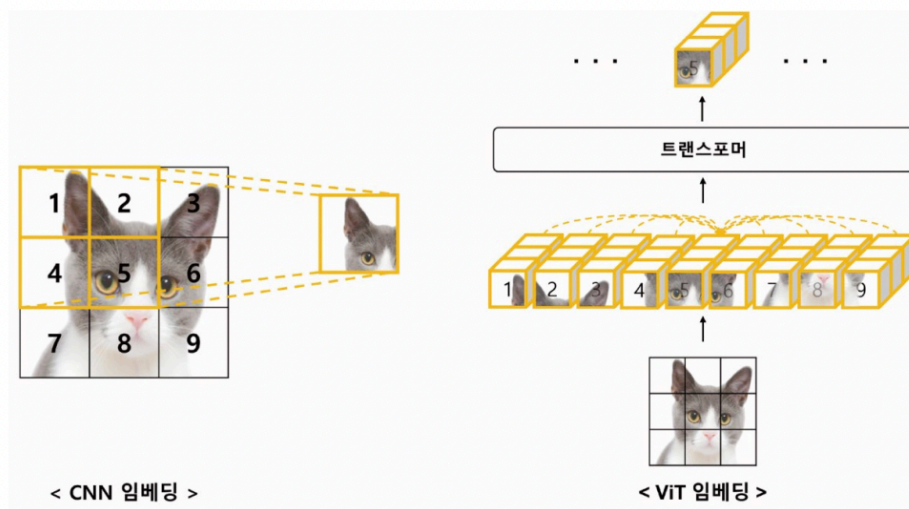


그림 10.3 합성곱 신경망과 ViT 계층의 차이

- 합성곱 신경망: 고양이 오른쪽 눈 표현할때 1, 2, 4, 5만 학습
- ViT 임베딩: 모든 이미지 패치가 학습에 관여
- ⇒ 좁은 수용 영역(Receptive Field) 가진 합성곱 신경망은 전체 이미지 정보 표현하는데 수많은 계층 필요

vs

트랜스포머 모델은 어텐션 거리 계산하여 1개의 ViT 레이어로 전체 이미지 정보 표현

- ViT 장점) 패치 단위로 이미지 처리 → 작은 모델로도 높은 성능
- ViT 단점) 입력 이미지 크기 고정 → 크기 다른 이미지는 이미지 크기 맞추는 전처리 필요
- ViT 단점) 패치간의 상대적 위치 정보만 고려 → 이미지 변환 취약

ViT의 귀납적 편향

- 딥러닝의 귀납적 편향 = 일반화 성능 향상을 위한 모델의 가정(Assumption)
- ex) 이미지 데이터: 공간적 관계 잘 표현 → Local 편향을 가진 합성곱 신경망 모델 많이 사용
시계열 데이터: 시간적 관계 잘 표현 → Sequential 편향 가진 순환 신경망 모델 많이 사용
- ViT 모델: 입력 데이터의 다양한 Query, Key, Value의 임베딩 형태로 일반화된 관계 학습 → 귀납적 편향 거의 없음, 대요량 데이터에 대해 이미지 특징들의 관계 잘 학습

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

- 귀납적 편향: 모델이 가지는 구조, 매개변수들이 데이터에 적합한 가정을 하고 있음을 나타냄
 - 가정 올바름 → 모델은 더 적은 데이터로 높은 일반화 성능
 - 너무 강한 귀납적 편향 → 다른 유형의 데이터/관계를 표현하는 데 어려움 초래

ViT 모델

구성 = 패치 임베딩 + 인코더

- 패치 임베딩: 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눔 → 각 패치를 벡터 형태로 변환
- 인코더: 각 패치와의 관계 학습
- 패치 임베딩, 인코더 계층 → 이미지 특징 추출, 작업에 맞는 출력값으로 변환해 사용

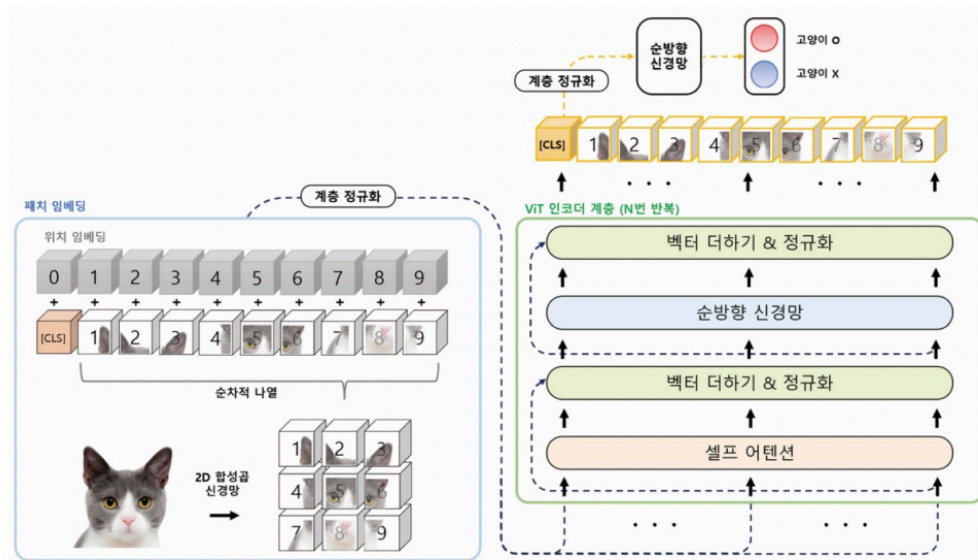


그림 10.4 ViT 모델 구조

패치 임베딩(Patch Embedding)

: 입력 이미지를 작은 패치로 분할하는 과정

- 작은 패치로 분할하기 위해서는 이미지 크기를 맞추는 전처리 필요
- ex) 입력 이미지 640x480 → 224×224 정방형
→ 전체 이미지를 패치 크기로 분할 → 시퀀셜 배열 만듦 (합성곱 신경망의 계층 활용)

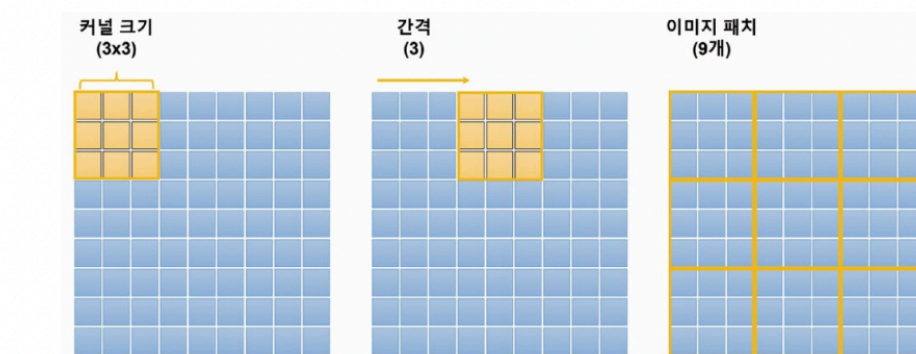


그림 10.5 이미지 패치 생성 과정

- Kernel Size와 Stride(간격) 설정 - 하이퍼파라미터로
 - kernel: 패치의 크기

- 간격: 패치가 이동하는 폭
- 이미지 크기 9x9, 커널 크기 3x3, 간격 3 → 총 9(3x3)개 이미지 패치 생성

수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9-3}{3} + 1 = 3$$

- 커널크기 3x3, 간격 1→ (9-3+1)x(9-3+1)=49개 패치 생성
- 이미지 패치 배열형태로 나열 - 원→오, 위→아래 순서로 배열
- 배열 가장 왼쪽에 분류 토큰(Special Classification Token) 추가
 - 분류토큰: 전체 이미지를 대표하는 벡터/ 특정 문제 예측
- 위치 임베딩(Position Embedding)→ 인접한 패치 간의 관계 학습
 - 패치의 위치 정보를 임베딩 벡터로 변환
 - 기존 이미지 패치 벡터들과 더함
- 마지막으로 계층 정규화 적용 → 패치 임베딩

인코더 계층

ViT 모델

- 이미지를 일정한 크기의 패치로 나뉘
- 각 패치를 벡터로 변환 후
- 인코더 계층에 입력으로 전달해
- 다양한 쿼리, 키, 값 임베딩의 관계를 학습

N개의 인코더 레이어 반복 적용

- 마지막 레이어: 분류 토큰 추출
- 분류 토큰 벡터: 이미지를 잘 표현하는 특징 벡터

ex) 고양이인지 아닌지 구분하는 모델 학습

시퀀셜 산출물(Sequential Output) 모두사용X, 분류 토큰 벡터만 사용

⇒ 분류 토큰 벡터를 순방향 신경망에 연결 → 고양이인지 아닌지 분류

모델 실습

colab

Swin Transformer

- 대규모 비전 인식 모델
- 기존 트랜스포머 문제점) 고정된 패치 크기 → 세밀한 예측 작업에 적합 X
셀프 어텐션 - 고해상도 이미지 처리하기 어려움
- 해결) 스윈 트랜스포머: 계층적 특징 맵 구성 → 1차 계산 복잡도
- 이미지 분류, 객체 감지, 영상 분할... 에서 강력한 성능
- 트랜스포머 구조보다 이미지 특성 더 잘 표현
- Shifted Window 기술 이용 → 입력 이미지를 일정한 크기의 패치로 분할 → 이 패치들을 쌓아 윈도우 구성 → 구성된 윈도우 영역만 셀프 어텐션 계산
 - 패치를 겹침 → 더 큰 효율성
 - 높은 해상도, 다양한 크기의 객체 효율적으로 처리

ViT와 스윈 트랜스포머 차이

ViT

- 다양한 이미지 크기/종횡비 가진 데이터 세트에 적용 어려움
- 이미지 공간 정보가 완전 보존X
- 대규모 데이터셋에서는 컴퓨팅 자원 많이 필요

→ 해결) 스윈 트랜스포머- local window로 물체의 크기/해상도 계층적 학습

- 로컬 윈도우: 입력 이미지 처리 위해 사용하는 고정 크기의 작은 윈도우

스윈 트랜스포머

- 로컬 윈도우 이동시키는 시프트 윈도우 기술 도입
 - 입력 이미지를 일정한 크기의 패치로 분할
 - 특징 맵 이동시켜 다음 계층에 사용
 - 패치의 위치를 더 유연하게 다룸
- ⇒ ViT보다 높은 정확도, 적은 매개변수, 넓은 범위의 이미지 크기/종횡비 다룸

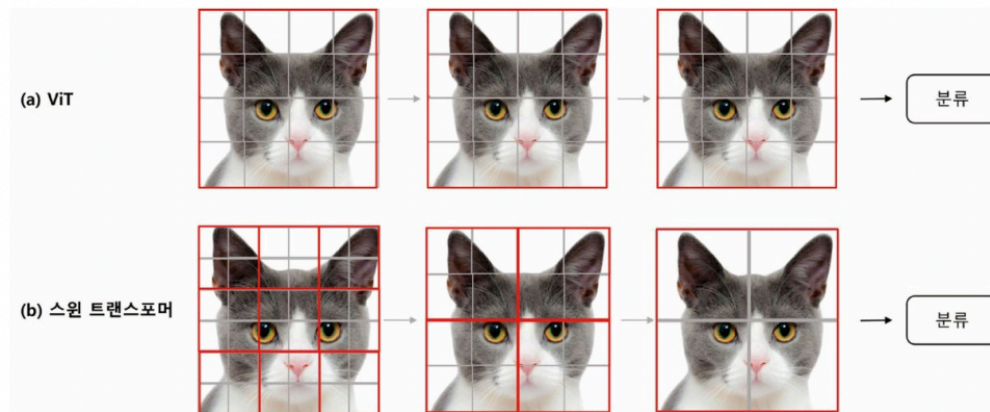


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

회색격자: 패치

빨간격자: 로컬 윈도우

ViT - 계층마다 전체 이미지 안에 어텐션이 수행되는 패치 크기/개수 동일

스윈 트랜스포머 - 로컬 윈도우로 계층마다 어텐션되는 패치 크기/개수를 다양하게 적용

- 입력 패치의 로컬 윈도우를 통해 계층적으로 접근
 - 시프트 윈도우로 로컬 윈도우 이동해가면서 인접 패치간 정보 계산
- ⇒ 이런 계층적 접근 방식
- 로컬 윈도우가 각 패치의 세부 정보에 집중
 - 시프트 윈도우를 통해 전역 특징 확인 가능 → 고해상도 이미지를 효율적으로 처리

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

스윈 트랜스포머 모델 구조

- 이미지를 패치 파티션으로 구분
 - 선형 임베딩 - 스윈 트랜스포머 블록 - 패치병합 연산 반복수행
1. 입력 이미지를 일정한 크기의 패치로 분할
 2. 각 패치에 대해 선형 임베딩 수행
→ 각 패치는 고정된 차원의 벡터로 변환
 3. 분할된 패치들을 기반으로 스윈 트랜스포머 블록 구성
 - 블록: 어텐션 계층, 계층 정규화, 다층 퍼셉트론 등 포함
 - 패치 간 상호작용 수행
 4. 패치를 병합해 전체 이미지에 대한 분류 수행
 - 분할된 패치들을 순차적으로 합치며 이미지의 전체적인 정보 추출

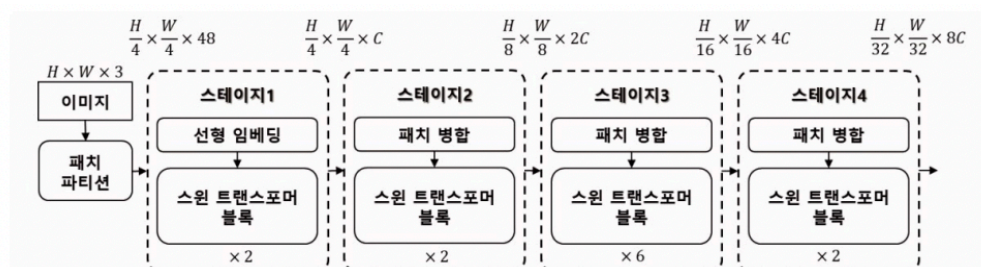


그림 10.7 스윈 트랜스포머 구조

- 네 개의 스테이지
- 각 스테이지는 서로 다른 해상도/패치 크기
- 선형 임베딩, 스윈 트랜스포머 블록, 패치 병합 등 반복 적용

- 스테이지 1: 패치들이 선형 임베딩 후 스윈 트랜스포머 블록에 전달됨
- 스테이지 2, 3, 4: 패치 병합 후 각 스윈 트랜스포머 블록에 전달
- 스윈 트랜스포머 블록: 윈도우 격자 안에 있는 패치 내부의 관계, 윈도우 격자 간의 관계 학습하는 두가지 어텐션 모듈로 구성

패치 파티션/ 패치 병합

패치 파티션(Patch Partition)

- 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식
- 분할 된 패치는 트랜스포머 계층의 입력으로 사용

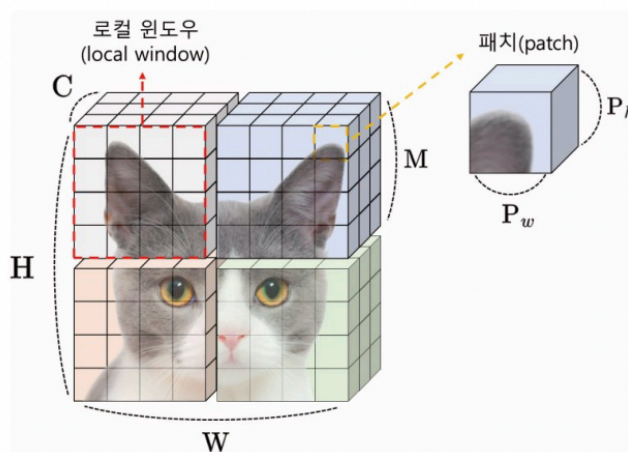


그림 10.8 패치 파티션 구조

일반적인 이미지 텐서 구조: $[C, H, W]$

- 패치 파티션을 통해 로컬 윈도우 추가
- M : 로컬 윈도우의 크기

패치 병합

- 인접한 패치들의 정보를 저차원으로 축소하는 과정
- 모델 매개변수의 수를 줄이기 위해
- $[C, H, W] \rightarrow [2C, H/2, W/2]$ 재정렬 $\rightarrow 2C$ 의 차원을 저차원으로 임베딩

스윈 트랜스포머 블록

- 패치 파티션 이후 네개의 스테이지 안에서 반복 적용
- 선형 임베딩 or 패치 병합 이후에 수행
- 계층 정규화, W-MSA, MLP, SW-MSA등으로 구성

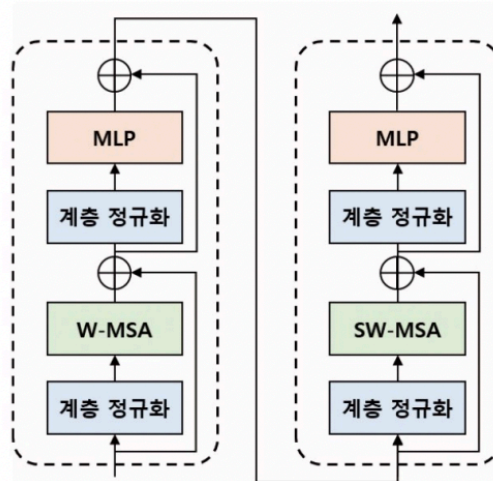


그림 10.9 스윈 트랜스포머 블록 구조

- W-MSA와 SW-MSA로 구성, 순차적으로 수행
- W-MSA: 로컬 윈도우를 사용하는 멀티 헤드 셀프 어텐션
 - 이미지에서 일정한 크기의 윈도우 영역 설정 → 해당 영역 내 픽셀 간의 어텐션을 계산
 - 연산량 대폭 감소, but 윈도우 내부의 이미지 패치 영역에만 셀프 어텐션을 수행해야 함

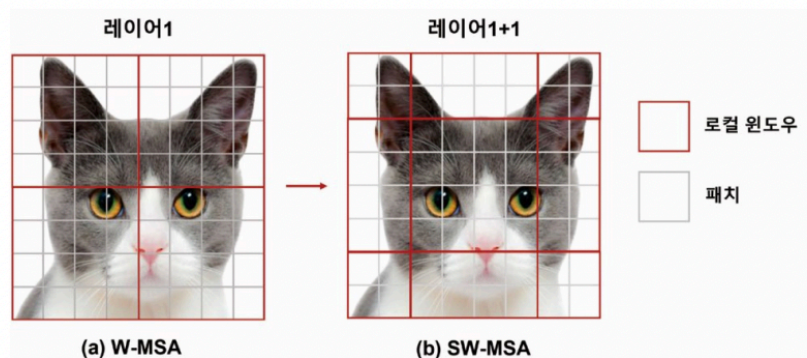


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- W-MSA: 효율적인 배치 계산으로 연산량 대폭 감소
 - 로컬 윈도우에서 연산을 수행하는 과정에도 배치축 사용 가능 → 연산 속도 빨라짐
 - 자연어 처리 작업에서 높은 성능
- SW-MSA: 더 많은 셀프 어텐션 연산 요구.
 - 해결) 순환 시프트와 어텐션 마스크 사용

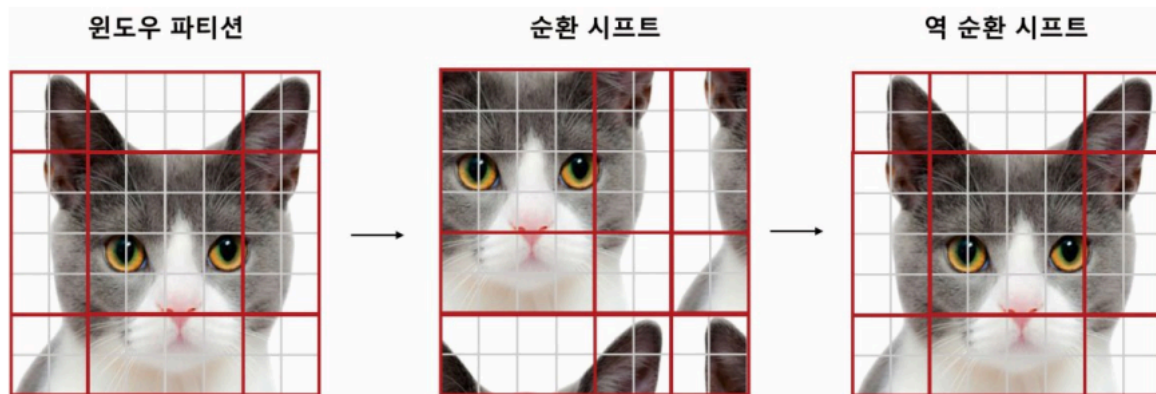


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

순환 시프트

- 로컬 윈도우가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져오는 것
- 로컬 윈도우 사이즈 M 보다 작은 $M/2$ 만큼 로컬 윈도우들 이동시킴
- 어텐션 마스크 사용 → 이동된 위치에서 연산 방지
- 순환 시프트 단계에서 셀프 어텐션 수행 → 역순환 시프트 사용: 원래 로컬 윈도우 구조로 복원
 - ⇒ SW-MSA 방식에서 로컬 윈도우 간의 셀프 어텐션 연산 효율적으로 수행

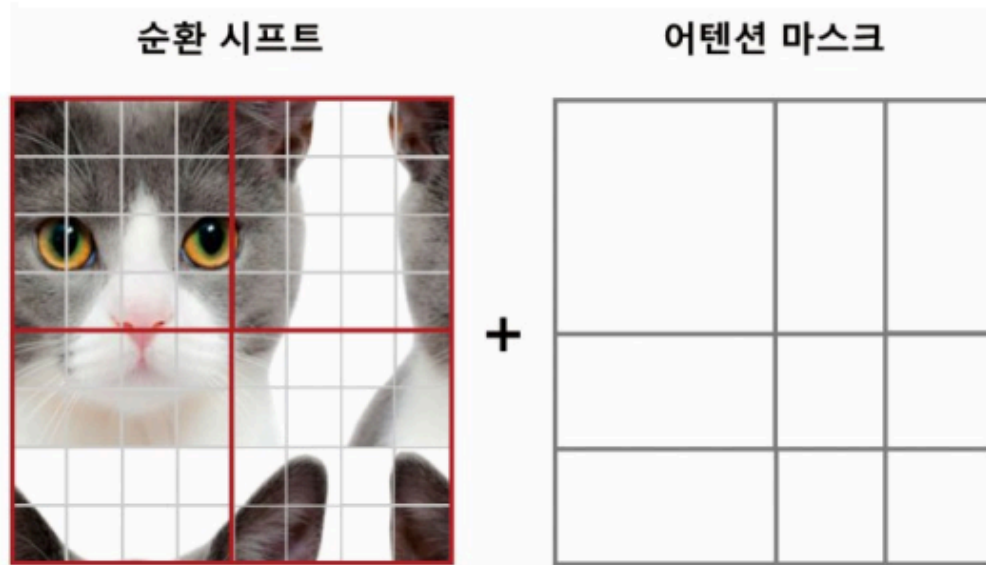


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

SW-MSWA

- 순환 시프트와 함께 어텐션 마스크 사용 → 로컬 윈도우 간의 셀프 어텐션 연산 효율적으로 수행
- 어텐션 마스크가, 순환 시프트를 통해 이동된 윈도우에 대한 정보 보존 & 불필요한 연산 줄이고 효율적인 계산 가능

기존 ViT와 다른 점

- 상대적 위치 편향(Relative Position Bias)고려
 - 목적: 로컬 윈도우 안에 있는 패치 간의 상대적인 거리를 임베딩



그림 10.13 상대적 위치 편향 계산 과정

if) 이미지 패치 1~4

- 패치 간의 상대적인 거리는 X, Y 축 두가지 방식으로 표기
- X축: 두패치 간의 가로 방향 거리
- Y축: 세로 방향 거리

예제 10.1 상대적 위치 편향 계산

`import torch`

`window_size=2`

`coords_h = torch.arange(window_size)`

`coords_w = torch.arange(window_size)`

`coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij"))`

`coords_flatten = torch.flatten(coords, 1)`

`relative_coords = coords_flatten[:, :, None] - coords_flatten[:, None, :]`

`print(relative_coords)`

`print(relative_coords.shape)`

- `window_size`: 여러개의 이미지 패치를 포함하는 격자의 크기
 - 윈도우 내외부의 패치를 구분하는 역할

- torch.meshgrid: coords_h 배열값과 coords_w 배열값
 - 사각형 격자(ij)만들때 사용
- torch.stack → (X, Y) 쌍 배열 생성 → torch.flatten → coords_flatten 생성 → 각 위치 색인의 차이 계산

...

CvT(Convolutional Vision Transformer)

- 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델
 - ⇒ local feature와 global feature을 모두 활용해 비교적 적은 수의 매개변수로 높은 성능
- 합성곱 계층의 지역 특징 추출, ViT의 셀프 어텐션 계층으로 전역 특징 결합해서 이미지 처

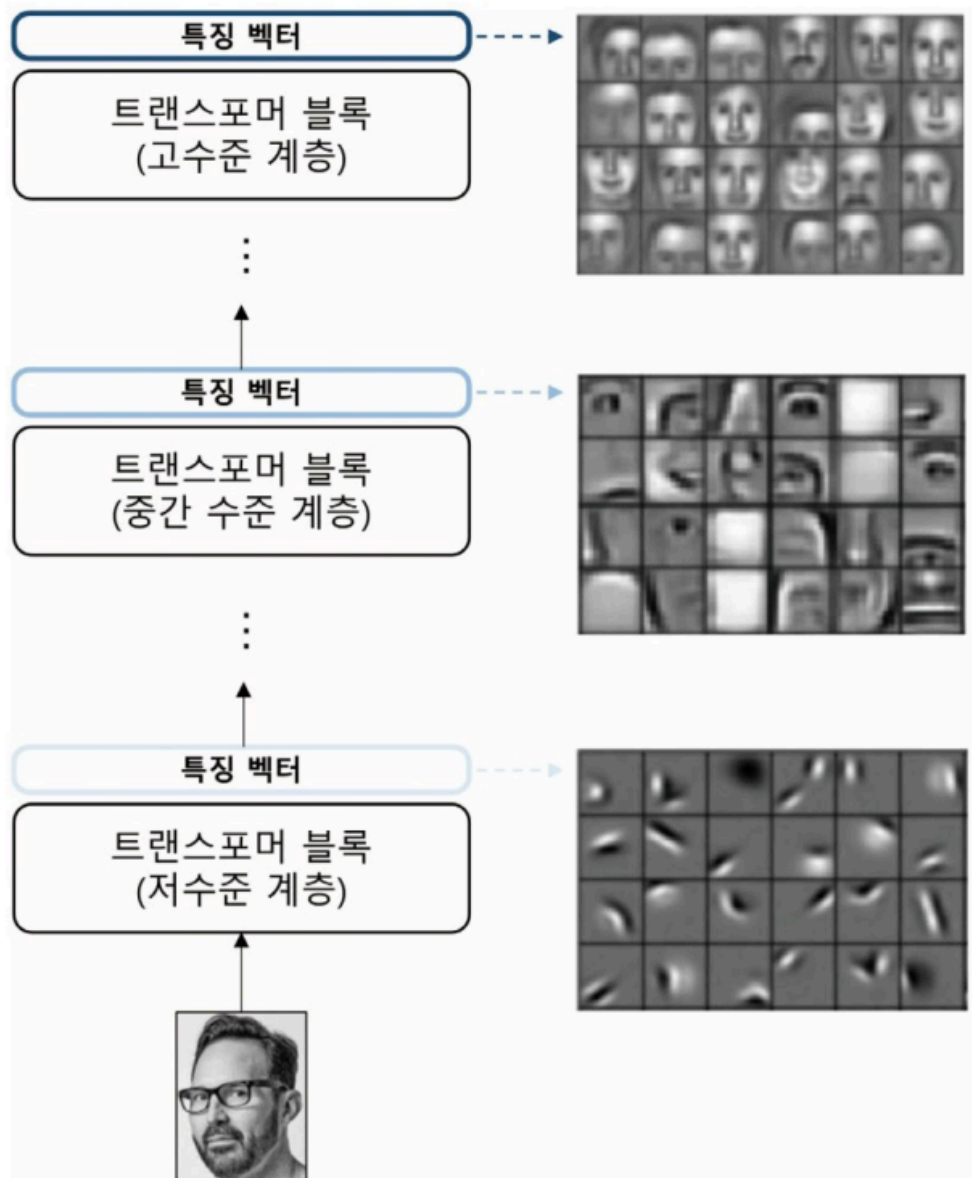


그림 10.14 CvT 모델의 계층적 특징 구조 예시

Low-level 계층: 선/점에 대한 특징 표현

Mid-level: 눈/귀 같은 특징 표현

High-level: 얼굴 구조에 대한 특징 표현

- 이미지 처리 전에 합성곱 연산으로 특징 맵 추출
→ 유의미한 패턴 추출, 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력으로 사용
- 입력되는 패치들을 겹쳐서 학습

표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

합성곱 토큰 임베딩 (Convolutional Token Embedding)

: 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩

- ViT에 계층적인 합성곱 신경망의 성질을 추가하기 위해 사용
- 이미지의 2D 구조를 보존하면서도 셀프 어텐션 계산
- 이미지 패치를 2D 합성곱 임베딩을 사용해 쿼리, 키, 값 임베딩으로 변환
- 변환된 임베딩을 사용해 셀프 어텐션 계산, 출력 생성

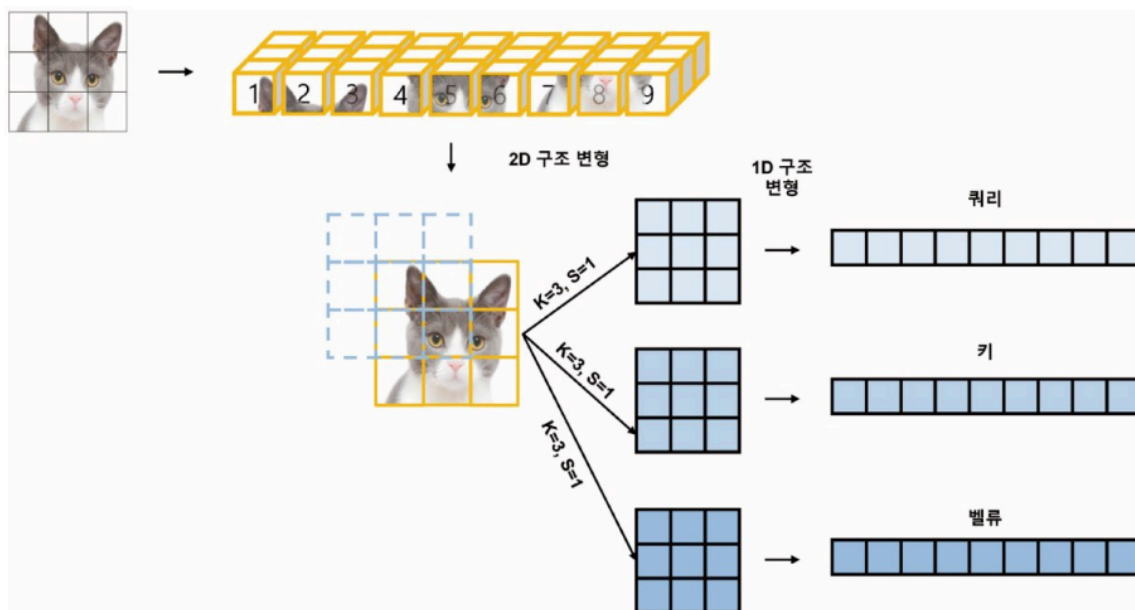


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

어텐션에 대한 합성곱 임베딩

: 2D로 재구성된 토큰맵에서 분리 가능한 합성곱 연산 적용해 성능 저하 및 계산 복잡도를 낮추는 연산

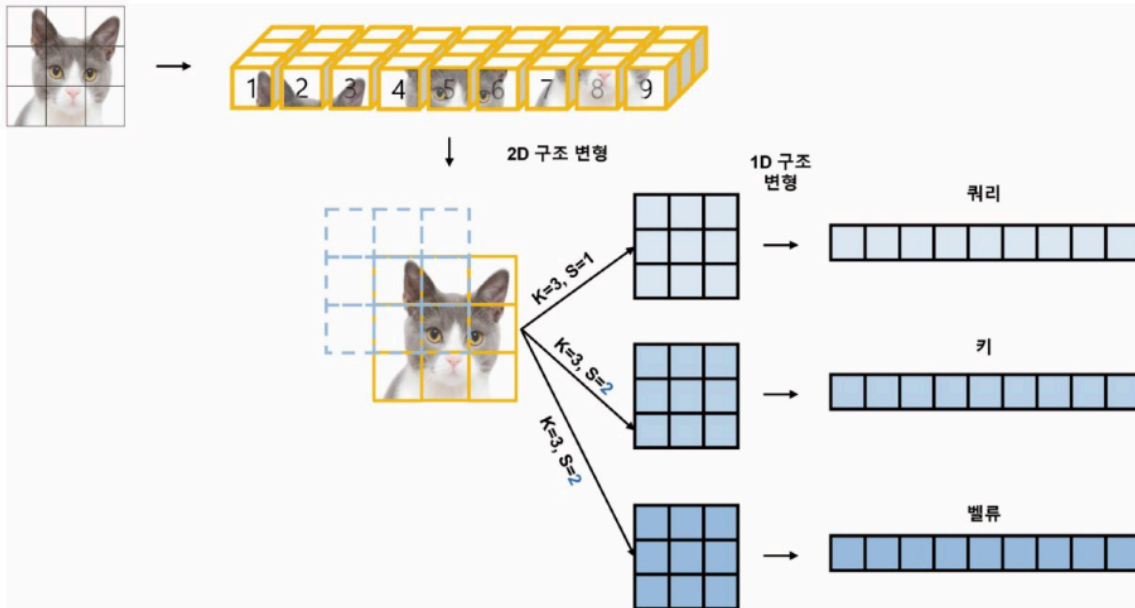


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

- 쿼리: 3×3 (K), 1 (S)의 커널 적용 $\rightarrow$ 9개의 이미지 패치 생성
- 키와 값: 3×3 (K), 2 (S)으로 커널 적용 $\rightarrow$ 4개의 이미지 패치 생성

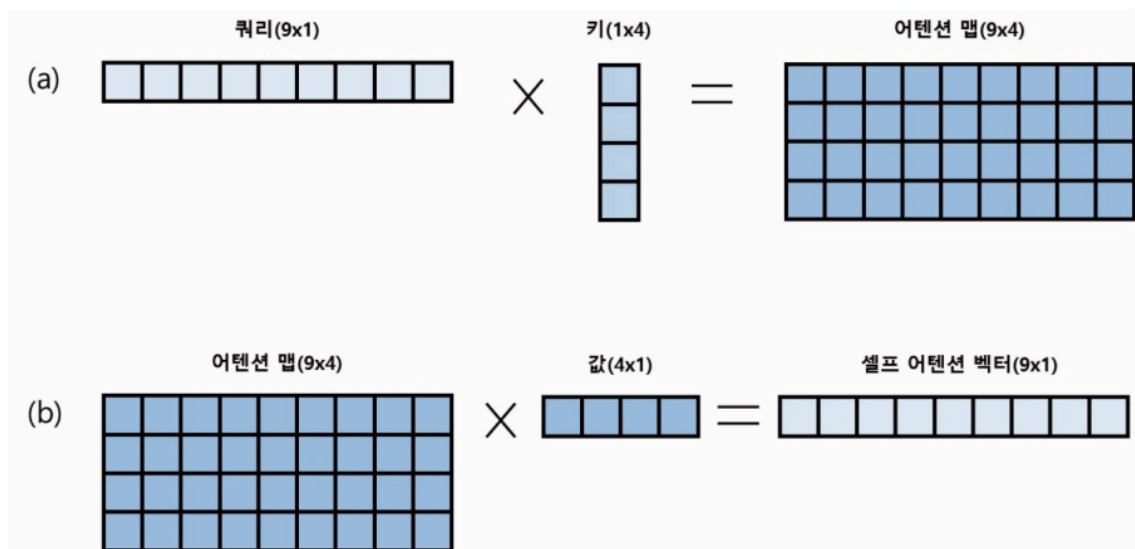


그림 10.17 축소된 셀프 어텐션 임베딩(열 $\times$ 행) 예시

9개 쿼리 패치 임베딩, 4개 키 패치 임베딩에 대한 중요도 - 어텐션 맵(Attention Map) 생성

$\rightarrow$ (b)처럼 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치 반영

- 트랜스포머 계층이 깊어질수록 벡터의 차원 증가 시킴 → 모델의 매개변수 수를 대폭 감소
- ex) 첫번째 트랜스포머 블록의 이미지 패치 64x64, 임베딩 차원 128, 마지막 트랜스포머 블록 8x8
 - 기존 임베딩 차원도 4배 증가 → 512차원
 - ⇒ 이미지 패치 크기가 줄어든 만큼 임베딩 차원을 늘리는 방법 선택 - 네트워크의 안정성 향상, 모델 설계 단순